

Machine Learning: Tutorial Questions with Solutions

Q1. Logistic regression is sometimes described as a *discriminative* classifier, while Gaussian Discriminant Analysis (GDA) is described as a *generative* classifier. Explain the distinction. Under what conditions might GDA outperform logistic regression, and when might logistic regression be preferred?

Solution. A **generative** classifier models the joint distribution $P(\mathbf{x}, y)$ by learning $P(\mathbf{x} | y)$ and $P(y)$, then applying Bayes' rule to obtain $P(y | \mathbf{x})$. GDA fits a Gaussian to each class and uses these to classify.

A **discriminative** classifier models $P(y | \mathbf{x})$ directly without modelling the input distribution. Logistic regression optimises the conditional likelihood directly.

When GDA is preferred: If the Gaussian class-conditional assumption holds (or approximately holds), GDA uses the data more efficiently and achieves lower error with fewer training examples. It can also handle missing features more naturally.

When logistic regression is preferred: When the Gaussian assumption is violated (e.g., binary or skewed features), logistic regression is more robust because it makes fewer assumptions. With large datasets, logistic regression often matches or exceeds GDA accuracy.

Q2. A Naïve Bayes model is trained on a text dataset in which the word “offer” never appears in any legitimate (non-spam) email. Describe exactly what goes wrong at test time when a legitimate email containing the word “offer” arrives, and explain how Laplace smoothing resolves the problem. Write the smoothed estimate $\hat{P}(\text{word} | y)$ explicitly.

Solution. Since “offer” never appears in the training set for class $y = 0$ (legitimate), the MLE gives $P(\text{offer} | y = 0) = 0$. At test time, the Naïve Bayes posterior for class 0 contains this factor:

$$P(y = 0) \prod_i P(x_i | y = 0) = P(y = 0) \times \cdots \times 0 \times \cdots = 0.$$

The entire product is zeroed out, so the classifier will *always* predict spam for any email containing “offer”, regardless of all other words — a failure called the **zero-frequency problem**.

Laplace smoothing adds a pseudocount of 1 to every word–class pair:

$$\hat{P}(\text{word} | y) = \frac{\text{count}(\text{word}, y) + 1}{\text{count}(y) + 2}.$$

This ensures every probability is strictly positive, eliminating the zero-product catastrophe while introducing only small bias when data is abundant.

Q3. Suppose you apply logistic regression to a two-class problem and plot the decision boundary. A colleague claims that “logistic regression always produces a straight-line decision boundary, so it cannot model non-linear class boundaries.” Is this claim correct? Explain, and describe one standard approach to overcome this limitation without changing the classifier itself.

Solution. The claim is **correct in the original feature space**: the logistic regression boundary is $\boldsymbol{\theta}^\top \mathbf{x} = 0$, which is a hyperplane (linear in $\mathbf{x}$).

However, the claim is **misleading as a fundamental limitation**. One can augment the feature vector with non-linear transformations of the original features — for example, polynomial features: replace (x_1, x_2) with $(x_1, x_2, x_1^2, x_2^2, x_1x_2)$. Logistic regression on the expanded feature space produces a boundary that is linear in the new features but *non-linear in the original input space* (e.g., an ellipse or parabola). This is called **feature engineering** or **basis expansion**, and it allows logistic regression to represent complex boundaries without modifying the classifier itself.

Q4. Consider the following training set for a binary classification problem:

x_1	x_2	y
1	1	1
1	0	0
0	1	0
0	0	0

Using Naïve Bayes with binary features, compute $P(y = 1 \mid x_1 = 1, x_2 = 0)$. Show all intermediate probabilities.

Solution. Priors: $P(y = 1) = 1/4$, $P(y = 0) = 3/4$.

Likelihoods (from the 4 examples):

For $y = 1$ (only the first row): $P(x_1 = 1 \mid y = 1) = 1$, $P(x_2 = 0 \mid y = 1) = 0$.

For $y = 0$ (rows 2–4): $P(x_1 = 1 \mid y = 0) = 1/3$, $P(x_2 = 0 \mid y = 0) = 2/3$.

Joint scores (unnormalised):

$$P(y = 1) P(x_1 = 1 \mid y = 1) P(x_2 = 0 \mid y = 1) = \frac{1}{4} \cdot 1 \cdot 0 = 0,$$

$$P(y = 0) P(x_1 = 1 \mid y = 0) P(x_2 = 0 \mid y = 0) = \frac{3}{4} \cdot \frac{1}{3} \cdot \frac{2}{3} = \frac{1}{6}.$$

Result: $P(y = 1 \mid x_1 = 1, x_2 = 0) = 0 / (0 + \frac{1}{6}) = 0$; the model predicts class 0.

Note: The zero arises because no training example with $y = 1$ has $x_2 = 0$. With Laplace smoothing, the result would be a small but positive probability.

Q5. A data point $\mathbf{x} = [4, 2]^\top$ is projected onto the first principal component $\mathbf{u}_1 = \frac{1}{\sqrt{5}}[2, 1]^\top$. Compute (a) the projection score $z = \mathbf{u}_1^\top \mathbf{x}$, (b) the reconstructed point $\hat{\mathbf{x}} = z \mathbf{u}_1$, and (c) the reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$.

Solution.

$$(a) \ z = \mathbf{u}_1^\top \mathbf{x} = \frac{1}{\sqrt{5}}(2 \cdot 4 + 1 \cdot 2) = \frac{10}{\sqrt{5}} = 2\sqrt{5}.$$

$$(b) \ \hat{\mathbf{x}} = z \mathbf{u}_1 = 2\sqrt{5} \cdot \frac{1}{\sqrt{5}}[2, 1]^\top = 2[2, 1]^\top = [4, 2]^\top.$$

$$(c) \ \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \|[4, 2]^\top - [4, 2]^\top\|^2 = 0.$$

The reconstruction error is zero because $\mathbf{x}$ lies exactly along the direction $\mathbf{u}_1$; no information is lost by keeping only this component.

Q6. Give two distinct failure modes of the K -means algorithm and, for each, suggest a concrete remedy.

Solution. **Failure mode 1 — Sensitivity to initialisation.** Poor initial centroids can trap the algorithm in a bad local minimum (K-means minimises a non-convex objective). *Remedy:* Use **K-means++** initialisation, which selects the first centroid uniformly at random and then chooses each subsequent centroid with probability proportional to its squared distance from the nearest already-chosen centroid. Alternatively, run K-means multiple times from different random starts and keep the solution with the lowest inertia.

Failure mode 2 — Inability to handle non-spherical or unequal clusters. K-means partitions space by Euclidean distance, so it implicitly assumes clusters are convex, roughly spherical, and of similar size. It fails on ring-shaped clusters, elongated clusters, or clusters of very different densities. *Remedy:* Use **DBSCAN** (density-based, handles arbitrary shapes) or **Gaussian Mixture Models** (soft assignment, elliptical clusters with learned covariances).

Q7. Describe the elbow method for selecting K in K -means. What quantity is plotted against K , why does it always decrease as K increases, and what does the elbow indicate?

Solution. The **elbow method** plots the **inertia** (within-cluster sum of squared distances, WCSS) as a function of K :

$$\text{WCSS}(K) = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2.$$

Why it always decreases: Adding one more cluster can only reduce or maintain WCSS, because K-means minimises WCSS and an extra cluster provides an additional degree of freedom (at the extreme, $K = n$ gives $\text{WCSS} = 0$).

The elbow is the value of K at which the rate of decrease changes sharply — the curve bends from steep to nearly flat. This is the point of diminishing returns: beyond the elbow, adding clusters yields only marginal reductions in WCSS at the cost of a more complex model. The elbow K balances compactness and model complexity.

Caveat: The elbow is not always sharp; the silhouette score or gap statistic can supplement this method.

Q8. A binary classifier on a heavily imbalanced dataset (95% negative, 5% positive) achieves 95% accuracy by predicting *negative* for every example. Explain why accuracy is a misleading metric here. Name two better metrics and give their formulas.

Solution. The trivial “always predict negative” classifier achieves 95% accuracy without learning anything useful; it has zero ability to detect the positive class. Accuracy is dominated by the majority class and fails to reflect performance on the minority (often more important) class.

Better metrics:

(1) **Precision** = $\frac{TP}{TP + FP}$: of all examples predicted positive, what fraction truly are? (Undefined/zero for the trivial classifier since it predicts no positives.)

(2) **Recall (Sensitivity)** = $\frac{TP}{TP + FN}$: of all true positives, what fraction were detected? (Zero for the trivial classifier.)

The **F1 score** = $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ combines both into a single metric and is also zero for the trivial classifier, correctly exposing its failure.

Q9. Consider a dataset in $\mathbb{R}^{200}$ (200 features, 1000 examples). The eigenvalues of the covariance matrix, in decreasing order, are $\lambda_1 = 60$, $\lambda_2 = 40$, $\lambda_3 = 25$, $\lambda_4 = 15$, with the remaining 196 eigenvalues summing to 60.

- (a) What is the total variance in the dataset?
- (b) What fraction of the variance is explained by the first three principal components?
- (c) How many components are needed to explain at least 70% of the variance?

Solution.

- (a) Total variance = $60 + 40 + 25 + 15 + 60 = \mathbf{200}$.
- (b) Variance from first 3 PCs = $60 + 40 + 25 = 125$. Fraction = $125/200 = \mathbf{62.5\%}$.
- (c) 70% of 200 = 140. Cumulative sums: 60 (30%), 100 (50%), 125 (62.5%), 140 (70%). Exactly 70% is reached at $k = 4$ components.

Q10. Dataset: $\{3, 5, 11, 13\}$, $K = 2$. Initial centroids: $\mu_1 = 4$, $\mu_2 = 12$. Carry out one complete iteration of K -means (assignment then update). State whether the algorithm has converged.

Solution. **Assignment step** (assign each point to the nearest centroid):

- $x = 3$: $|3 - 4| = 1 < |3 - 12| = 9 \Rightarrow$ cluster 1.
- $x = 5$: $|5 - 4| = 1 < |5 - 12| = 7 \Rightarrow$ cluster 1.
- $x = 11$: $|11 - 4| = 7 > |11 - 12| = 1 \Rightarrow$ cluster 2.
- $x = 13$: $|13 - 4| = 9 > |13 - 12| = 1 \Rightarrow$ cluster 2.

Clusters: $C_1 = \{3, 5\}$, $C_2 = \{11, 13\}$.

Update step:

$$\mu_1 = (3 + 5)/2 = 4, \quad \mu_2 = (11 + 13)/2 = 12.$$

The centroids are unchanged, so the algorithm has **converged** after one iteration.

Q11. A decision tree node contains 12 training examples: 7 of class A and 5 of class B. A candidate split produces a left child with 6 examples (all class A) and a right child with 6 examples (1 class A, 5 class B).

- (a) Compute the Gini impurity of the parent node.

- (b) Compute the weighted Gini impurity after the split.
- (c) Compute the reduction in impurity and state whether this is a good split.

Solution.

(a) $G_{\text{parent}} = 1 - \left[\left(\frac{7}{12} \right)^2 + \left(\frac{5}{12} \right)^2 \right] = 1 - \frac{49+25}{144} = \frac{70}{144} \approx \mathbf{0.486}.$

(b) Left child (pure): $G_L = 0$. Right child: $G_R = 1 - \left[\left(\frac{1}{6} \right)^2 + \left(\frac{5}{6} \right)^2 \right] = \frac{10}{36} \approx 0.278.$

Weighted: $\frac{6}{12} \cdot 0 + \frac{6}{12} \cdot 0.278 = \mathbf{0.139}.$

- (c) Reduction = $0.486 - 0.139 = \mathbf{0.347}$. This is a large reduction relative to the parent impurity. The split is **very good**: one child is perfectly pure and the other is strongly skewed toward class B.

Q12. A decision tree achieves 100% training accuracy but 60% test accuracy. Name this phenomenon and describe two strategies—one that operates *before* the tree is fully grown and one that operates *after*—to mitigate it.

Solution. This is **overfitting**: the tree has memorised the training data (including noise) rather than learning the underlying pattern.

Pre-pruning (stopping criteria applied during growth):

- Set a *maximum depth*: the tree is not allowed to grow beyond a specified number of levels.
- Set a *minimum samples per split*: refuse to split a node if it contains fewer than some threshold of examples.
- Require a *minimum information gain*: only split if the gain exceeds $\varepsilon > 0$.

Post-pruning (applied after the tree is fully grown):

- *Reduced-error pruning*: working bottom-up, remove each subtree if doing so does not decrease accuracy on a held-out validation set.
- *Cost-complexity pruning* (CART): minimise a penalised objective $R_\alpha(T) = R(T) + \alpha|T|$ where $|T|$ is the number of leaves, selecting α via cross-validation.

Q13. In Gaussian Discriminant Analysis with a shared covariance matrix Σ , the parameters ϕ , μ_0 , μ_1 , and Σ are estimated by maximum likelihood. Write down the closed-form MLE for each of the four quantities (you do not need to derive them, just state the formulas).

Solution. Let n be the total number of training examples and $n_k = \sum_i \mathbf{1}[y^{(i)} = k]$ the count of class- k examples.

$$\hat{\phi} = \frac{n_1}{n}$$

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i: y^{(i)}=k} \mathbf{x}^{(i)}, \quad k \in \{0, 1\}$$

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - \hat{\mu}_{y^{(i)}})(\mathbf{x}^{(i)} - \hat{\mu}_{y^{(i)}})^\top$$

The shared $\hat{\Sigma}$ is computed by pooling all examples (subtracting the appropriate class mean from each), giving the *within-class* covariance matrix.

Q14. Two classifiers are compared on the same binary classification task. Classifier A has precision 0.80 and recall 0.60. Classifier B has precision 0.65 and recall 0.85. (a) Compute the F1 score of each. (b) In a medical diagnosis application where missing a positive case is very costly, which classifier is preferable, and why?

Solution.

$$(a) F_1(A) = \frac{2 \times 0.80 \times 0.60}{0.80 + 0.60} = \frac{0.96}{1.40} \approx \mathbf{0.686}.$$

$$F_1(B) = \frac{2 \times 0.65 \times 0.85}{0.65 + 0.85} = \frac{1.105}{1.50} \approx \mathbf{0.737}.$$

- (b) **Classifier B** is preferable. Missing a positive case (a false negative) is very costly, so high **recall** is paramount. Classifier B achieves recall 0.85 vs. A's 0.60, catching far more true positive cases. The cost of lower precision (more false alarms) is acceptable if the follow-up cost of a false positive is much lower than the cost of a missed diagnosis.

Q15. State the gradient descent update rule for logistic regression (without regularisation). Define every symbol you use. Explain why the cross-entropy loss is preferred over squared error for training a logistic regression model.

Solution. The model predicts $\hat{y}^{(i)} = \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})$ where $\sigma(z) = 1/(1 + e^{-z})$.

The cross-entropy loss is $J(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$.

Gradient descent update for each parameter θ_j :

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial J}{\partial \theta_j}, \quad \frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}.$$

Symbols: η = learning rate; $y^{(i)} \in \{0, 1\}$ = true label; $x_j^{(i)}$ = j -th feature of example i ; n = training set size.

Why not squared error? With squared error and the sigmoid, the loss surface is non-convex and contains many local minima that trap gradient descent. Cross-entropy combined with the sigmoid yields a *convex* loss in $\boldsymbol{\theta}$, guaranteeing a unique global minimum. Cross-entropy is also the natural loss for Bernoulli-distributed outputs, arising directly from maximum likelihood estimation.

Q16. Suppose you apply PCA to a dataset and retain the top k components. Describe two distinct criteria a practitioner might use to choose k , and give one practical consideration that might lead you to choose a *smaller* k than the criteria suggest.

Solution. **Criterion 1 — Explained variance threshold:** Choose the smallest k such that

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i} \geq \tau$$

for some threshold τ (commonly 90% or 95%). This ensures a prescribed fraction of information is retained.

Criterion 2 — Scree plot / elbow: Plot eigenvalues in decreasing order and look for the point where they drop sharply and then level off. Components before the elbow capture most of the structure; those after capture mostly noise.

Practical reason to choose smaller k : Computational or memory constraints may require a smaller representation for deployment. Retaining fewer components also acts as regularisation — adding noisy components (even those above the threshold) can hurt generalisation of a downstream model. Interpretability is another consideration: fewer components may allow the analyst to understand what each component represents.

Q17. Describe the relationship between GDA and logistic regression. Specifically: if the GDA modelling assumptions hold exactly, which classifier achieves lower asymptotic error, and why? What happens when the assumptions do not hold?

Solution. Both GDA (with shared Gaussian class-conditionals) and logistic regression yield a posterior of the form $P(y = 1 \mid \mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$ for some $\boldsymbol{\theta}$. GDA *implies* logistic regression: any GDA model corresponds to a logistic regression model. The converse is not true — logistic regression is a strictly larger model class.

When GDA assumptions hold exactly: GDA is asymptotically more efficient — it achieves lower error with fewer training examples because, by modelling $P(\mathbf{x} \mid y)$, it extracts more information from the data under a correct model.

When assumptions fail: Logistic regression is more robust. Because it only models $P(y \mid \mathbf{x})$ directly, it is not harmed by incorrect assumptions about $P(\mathbf{x} \mid y)$ (e.g., non-Gaussian, binary, or skewed features). In practice, with large datasets, logistic regression often matches or surpasses GDA even when the Gaussian assumption is approximately satisfied.

Q18. A dataset has $n = 6$ points. A bootstrap sample of size 6 is drawn with replacement.

- (a) What is the probability that a specific point $x^{(i)}$ is *not* included in the bootstrap sample?
- (b) Approximately what fraction of the original training data is excluded on average? What are these excluded points called, and how are they used?

Solution.

- (a) At each of the 6 draws, the probability of not selecting $x^{(i)}$ is $1 - 1/6 = 5/6$. Over 6 independent draws:

$$P(x^{(i)} \text{ not selected}) = \left(\frac{5}{6}\right)^6 = \frac{15625}{46656} \approx \mathbf{0.335}.$$

- (b) On average, approximately **33.5%** of the original examples are excluded from each bootstrap sample (this approaches $1/e \approx 36.8\%$ as $n \rightarrow \infty$). These excluded examples are called **out-of-bag (OOB) examples**. They serve as a built-in validation set: each tree is evaluated on its own OOB examples, providing an

unbiased estimate of generalisation error without needing a separate held-out split.

Q19. A regression tree is to be fit to the following five points:

$$(x, y) \in \{(1, 3), (2, 5), (3, 4), (8, 9), (9, 11)\}.$$

Consider a split at $x = 5$ (left: $x \leq 5$, right: $x > 5$). Compute the weighted sum of squared errors (SSE) after this split and compare it to the SSE before splitting (treating all five points as one node).

Solution. Before splitting (one node): $\bar{y} = (3 + 5 + 4 + 9 + 11)/5 = 6.4$.

$$\begin{aligned}\text{SSE}_{\text{total}} &= (3 - 6.4)^2 + (5 - 6.4)^2 + (4 - 6.4)^2 + (9 - 6.4)^2 + (11 - 6.4)^2 \\ &= 11.56 + 1.96 + 5.76 + 6.76 + 21.16 = \mathbf{47.2}.\end{aligned}$$

After splitting at $x = 5$:

$$\text{Left } \{(1, 3), (2, 5), (3, 4)\}: \bar{y}_L = 4. \quad \text{SSE}_L = (3 - 4)^2 + (5 - 4)^2 + (4 - 4)^2 = 2.$$

$$\text{Right } \{(8, 9), (9, 11)\}: \bar{y}_R = 10. \quad \text{SSE}_R = (9 - 10)^2 + (11 - 10)^2 = 2.$$

$$\text{Weighted SSE} = \frac{3}{5} \cdot 2 + \frac{2}{5} \cdot 2 = 1.2 + 0.8 = \mathbf{2.0}.$$

Reduction $= 47.2 - 2.0 = 45.2$ (a 95.8% reduction). This is an excellent split, consistent with the clear gap between $x \leq 3$ and $x \geq 8$ in the data.

Q20. The Naïve Bayes assumption states that features are conditionally independent given the class label. Give a realistic example where this assumption is clearly violated. Despite the violated assumption, Naïve Bayes often performs surprisingly well in practice. Provide one reason why.

Solution. Example of violated assumption: In an email spam classifier, the words “free” and “offer” are *not* conditionally independent given the class spam: emails using “free” are much more likely to also use “offer.” There is strong positive correlation between these features within the spam class, violating the independence assumption.

Why Naïve Bayes often works despite this: Naïve Bayes does not need to estimate $P(y \mid \mathbf{x})$ accurately — it only needs to *rank* classes correctly (identify the argmax). Even if the probability values are poorly calibrated due to the independence violation, the *direction* of the posterior is often correct. Correlated features contribute redundant but not harmful evidence, and the argmax prediction is frequently still the right class. Additionally, the model has far fewer parameters than the full joint model, so it does not overfit even with limited data.